

SQL NOTES

* 1. WHAT IS SQL?

SQL (Structured Query Language) is used to communicate with databases. It is used to store, retrieve, update and manage data in relational databases.

2. TYPES OF SQL COMMANDS

- **DDL** - Data Definition Language (CREATE, ALTER, DROP, TRUNCATE)
- **DML** - Data Manipulation Language (INSERT, UPDATE, DELETE)
- **DQL** - Data Query Language (SELECT)
- **DCL** - Data Control Language (GRANT, REVOKE)
- **TCL** - Transaction Control Language (COMMIT, ROLLBACK, SAVEPOINT)

3. BASIC QUERIES

- `SELECT * FROM table_name;` -- get all columns
- `SELECT column1, column2 FROM table_name;` -- get specific columns
- `SELECT * FROM table_name WHERE condition;` -- filter records
- `SELECT * FROM table_name ORDER BY column ASC|DESC;` -- sort results
- `SELECT * FROM table_name LIMIT n;` -- limit rows

4. CLAUSES

- WHERE** - filters rows based on condition
- ORDER BY** - sorts the result set
- GROUP BY** - groups rows that have same values
- HAVING** - filters groups based on condition
- LIMIT** - limits number of rows
- DISTINCT** - returns unique values

5. AGGREGATE FUNCTIONS

FUNCTION	DESCRIPTION
<code>COUNT()</code>	- counts number of rows
<code>SUM()</code>	- returns sum of values
<code>AVG()</code>	- returns average
<code>MIN()</code>	- returns minimum value
<code>MAX()</code>	- returns maximum value

6. JOINS

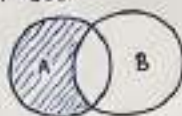
- **INNER JOIN**



Returns only matching rows from both tables.

```
SELECT * FROM A
INNER JOIN B ON A.id = B.id;
```

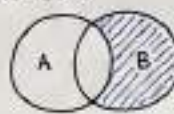
- **LEFT JOIN**



Returns all rows from left table and matching rows from right table.

```
SELECT * FROM A
LEFT JOIN B ON A.id = B.id;
```

- **RIGHT JOIN**



Returns all rows from right table and matching rows from left table.

```
SELECT * FROM A
RIGHT JOIN B ON A.id = B.id;
```

- **FULL JOIN**



Returns all rows when there is a match in either left or right table.

```
SELECT * FROM A
FULL JOIN B ON A.id = B.id;
```

7. GROUP BY and HAVING

```
SELECT column, COUNT(*)
FROM table_name
GROUP BY column
HAVING COUNT(*) > 1;
```

-- groups by column
-- filter groups

8. SUBQUERIES

```
SELECT name
FROM students
WHERE id IN (SELECT student_id
FROM marks
WHERE marks > 80);
```

-- inner query executed first

9. CONSTRAINTS

- **PRIMARY KEY** - unique identifier for a row
- **FOREIGN KEY** - links to primary key in another table
- **NOT NULL** - column cannot have NULL values
- **UNIQUE** - all values in column must be unique
- **CHECK** - ensures values satisfy a condition
- **DEFAULT** - sets default value for a column

10. SAMPLE TABLE

STUDENTS TABLE

id(PK)	name	age	course
1	Vamsee	21	CSE
2	Riya	20	ECE
3	Arjun	22	IT

EXAMPLE QUERY:

```
SELECT name, age
FROM students
WHERE age > 20
ORDER BY age DESC;
```

11. SET OPERATORS

- UNION - combines result of two queries (removes duplicates)
- UNION ALL - combines result (keeps duplicates)
- INTERSECT - returns only common rows
- EXCEPT - returns rows from first query that are not in second query

```
SELECT column_name FROM table1
UNION
SELECT column_name FROM table2;
```

-- must have same number of columns and compatible data types

12. ALIAS (AS)

- Used to give a temporary name to a table or column.

-- column alias

```
SELECT name AS student_name FROM students;
```

-- table alias

```
SELECT s.name FROM students AS s;
```

13. ORDER BY

- Used to sort the result set.

```
SELECT name, age FROM students
ORDER BY age ASC;
```

-- ASC: ascending
-- DESC: descending

14. NULL VALUES

- NULL means unknown or missing value.
- Comparisons with NULL must use IS NULL / IS NOT NULL.

```
SELECT * FROM students
WHERE age IS NULL;
```

-- rows where age is null

```
SELECT * FROM students
WHERE age IS NOT NULL;
```

-- rows where age is not null

15. WILDCARD OPERATORS

- LIKE - pattern matching
- % - represents zero or more characters
- _ - represents single character

```
SELECT * FROM students WHERE name LIKE 'A%'; -- names starting with A
SELECT * FROM students WHERE name LIKE '%e'; -- names ending with e
SELECT * FROM students WHERE name LIKE '_a%'; -- second letter is a
```

16. BETWEEN, IN, NOT IN

- BETWEEN - selects values within a range
- IN - checks if value matches any in list
- NOT IN - checks if value does not match any in list

```
SELECT * FROM students WHERE age BETWEEN 18 AND 25;
SELECT * FROM students WHERE course IN ('CS', 'ECE', 'IT');
SELECT * FROM students WHERE course NOT IN ('Mechanical');
```

17. LIMIT & OFFSET

- LIMIT - limits number of rows returned
- OFFSET - skips a number of rows

```
SELECT * FROM students LIMIT 5; -- first 5 rows
SELECT * FROM students LIMIT 5 OFFSET 5; -- next 5 rows
```

18. CASE STATEMENT

- Used for conditional logic in queries.

```
SELECT name, age,
CASE
  WHEN age >= 18 THEN 'Adult'
  WHEN age >= 13 THEN 'Teenager'
  ELSE 'Child'
END AS age_group
FROM students;
```

-- returns age group based on condition

19. SUBSTRING, LENGTH, UPPER, LOWER

- Common string functions.

FUNCTION	DESCRIPTION	EXAMPLES
SUBSTRING(str, start, len)	- extracts substring	SELECT SUBSTRING(name, 1, 3) FROM students;
LENGTH(str)	- returns length	SELECT LENGTH(name) FROM students;
UPPER(str)	- converts to upper case	SELECT UPPER(name) FROM students;
LOWER(str)	- converts to lower case	SELECT LOWER(name) FROM students;

20. AGGREGATE FUNCTIONS WITH GROUP BY

- Aggregate functions are used with GROUP BY to get summary for each group.

```
SELECT course, COUNT(*) AS total_students,
AVG(age) AS avg_age
FROM students
GROUP BY course;
```

-- summary per course

21. HAVING CLAUSE

- Used to filter groups based on aggregate condition.

```
SELECT course, COUNT(*) AS total_students
FROM students
GROUP BY course
HAVING COUNT(*) > 10;
```

-- only groups having more than 10 students

22. RANK, DENSE_RANK, ROW_NUMBER

- Window (Analytical) functions.

FUNCTION	DESCRIPTION	EXAMPLE
RANK()	- same rank, leaves gaps	SELECT name, marks, RANK() OVER (ORDER BY marks DESC) AS rank_no FROM students;
DENSE_RANK()	- same rank, no gaps	
ROW_NUMBER()	- unique sequential number for each row	

23. PARTITION BY (with Window Functions)

- Used to divide data into partitions before applying window function.

```
SELECT name, course, marks,
RANK() OVER (PARTITION BY course ORDER BY marks DESC)
AS rank_in_course
FROM students;
```

-- rank within each course

24. VIEW (VIRTUAL TABLE)

- A view is a stored query.
- It acts like a virtual table.

```
CREATE VIEW view_students AS
SELECT id, name, course FROM students;
```

```
SELECT * FROM view_students;
```

-- use view like a table

25. INDEX

- Improves performance of queries.
- Creates a sorted structure for quick search.

```
CREATE INDEX idx_name ON students(name); -- create index
DROP INDEX idx_name; -- drop index
```

26. ALTER, DROP, TRUNCATE

- ALTER - modifies structure
- DROP - removes table completely
- TRUNCATE - removes all rows but keeps structure

```
ALTER TABLE students ADD email VARCHAR(50); -- add column
ALTER TABLE students DROP COLUMN email; -- drop column
DROP TABLE students; -- delete table
TRUNCATE TABLE students; -- delete all rows
```

27. USEFUL TIPS

1. SQL keywords are NOT case sensitive (SELECT = select).
2. Always end query with a semicolon (;).
3. Use meaningful names for tables and columns.
4. Practice with different queries to build logic.

Practice Makes Perfect!



28. RENAME

- Used to rename objects.

```
ALTER TABLE students RENAME TO student_info; -- rename table
ALTER TABLE students RENAME COLUMN age TO student_age; -- rename column
```

29. DEFAULT

- Sets a default value for a column.

```
ALTER TABLE students
ADD COLUMN status VARCHAR(20) DEFAULT 'Active'; -- default value
```

30. CHECK CONSTRAINT

- Ensures that values in a column satisfy a condition.

```
ALTER TABLE students
ADD CONSTRAINT chk_age CHECK (age >= 10); -- age must be >= 10
```

31. UNIQUE CONSTRAINT

- Ensures all values in a column are different.

```
ALTER TABLE students
ADD CONSTRAINT uk_email UNIQUE (email); -- email must be unique
```

32. PRIMARY KEY

- Uniquely identifies each row in a table.

```
ALTER TABLE students
ADD CONSTRAINT pk_student PRIMARY KEY (id); -- id is primary key
```

33. FOREIGN KEY

- Links data between two tables.

```
ALTER TABLE orders
ADD CONSTRAINT fk_student
FOREIGN KEY (student_id)
REFERENCES students(id); -- student_id in orders refers to students.id
```

34. CREATE TABLE (Example)

- Creates a new table.

```
CREATE TABLE students (
  id INT PRIMARY KEY,
  name VARCHAR(50) NOT NULL,
  age INT CHECK (age >= 10),
  email VARCHAR(50) UNIQUE,
  course VARCHAR(50) DEFAULT 'CSE',
  status VARCHAR(20) DEFAULT 'Active'
); -- example with constraints
```

35. INSERT INTO

- Adds new records into a table.

```
INSERT INTO students (id, name, age, email, course)
VALUES (1, 'Vamsi', 21, 'vamsi@example.com', 'CSE');
```

36. UPDATE

- Modifies existing records.

```
UPDATE students
SET age = 22, course = 'ECE'
WHERE id = 1; -- update specific record
```

37. DELETE

- Removes records from a table.

```
DELETE FROM students
WHERE id = 1; -- delete specific record
```

38. TRUNCATE TABLE

- Removes all records from a table (faster than DELETE).

```
TRUNCATE TABLE students; -- remove all rows
```

39. JOINS SUMMARY

JOIN TYPE	DESCRIPTION	SYNTAX EXAMPLE	RETURNS
INNER JOIN	Returns only matching rows	SELECT * FROM A INNER JOIN B ON A.id = B.id;	Matching rows from both tables
LEFT JOIN	Returns all rows from left table	SELECT * FROM A LEFT JOIN B ON A.id = B.id;	All from left, matching from right
RIGHT JOIN	Returns all rows from right table	SELECT * FROM A RIGHT JOIN B ON A.id = B.id;	All from right, matching from left
FULL JOIN	Returns all rows from both tables	SELECT * FROM A FULL JOIN B ON A.id = B.id;	All rows from both tables
CROSS JOIN	Returns Cartesian product	SELECT * FROM A CROSS JOIN B;	All possible combinations

40. SET OPERATIONS SUMMARY

OPERATION	DESCRIPTION	SYNTAX EXAMPLE	NOTES
UNION	Combines results and removes duplicates	SELECT col FROM A UNION SELECT col FROM B;	Columns must be compatible
UNION ALL	Combines results and keeps duplicates	SELECT col FROM A UNION ALL SELECT col FROM B;	Faster than UNION
INTERSECT	Returns common rows	SELECT col FROM A INTERSECT SELECT col FROM B;	Returns only common rows
EXCEPT	Returns rows from first not in second	SELECT col FROM A EXCEPT SELECT col FROM B;	Some DBs use MINUS

39. LIKE OPERATOR

- Searches for a pattern in a column.

```
SELECT * FROM students WHERE name LIKE 'A%'; -- starts with A
SELECT * FROM students WHERE name LIKE '%a'; -- ends with a
SELECT * FROM students WHERE name LIKE '%am%'; -- contains 'am'
SELECT * FROM students WHERE name LIKE '_a%'; -- second letter is a
```

40. IN OPERATOR

- Checks if a value matches any value in a list.

```
SELECT * FROM students
WHERE course IN ('CSE', 'ECE', 'IT'); -- course is CSE or ECE or IT
```

41. BETWEEN OPERATOR

- Checks if a value is within a range.

```
SELECT * FROM students
WHERE age BETWEEN 18 AND 25; -- age between 18 and 25
```

42. IS NULL / IS NOT NULL

- Checks for NULL values.

```
SELECT * FROM students WHERE email IS NULL; -- email is null
SELECT * FROM students WHERE email IS NOT NULL; -- email is not null
```

43. DISTINCT KEYWORD

- Returns only unique values.

```
SELECT DISTINCT course FROM students; -- unique courses
```

44. GROUP BY (Example)

- Groups rows that have the same values.

```
SELECT course, COUNT(*) AS total_students
FROM students
GROUP BY course; -- count of students in each course
```

45. HAVING (Example)

- Filters groups created by GROUP BY.

```
SELECT course, COUNT(*) AS total_students
FROM students
GROUP BY course
HAVING COUNT(*) > 5; -- only courses with more than 5 students
```

46. ORDER BY (Example)

- Sorts the result.

```
SELECT * FROM students
ORDER BY age ASC; -- sort by age ascending
```

47. LIMIT & OFFSET (Example)

- Limits the result set and skips rows.

```
SELECT * FROM students LIMIT 10 OFFSET 5; -- skip 5 rows, next 10 rows
```

48. SUBQUERY (Example)

- A query inside another query.

```
SELECT name FROM students
WHERE id IN (SELECT student_id FROM orders); -- students who have orders
```

KEEP
PRACTICING
SQL! ☺

51. UNION vs UNION ALL

- UNION removes duplicates.
- UNION ALL keeps duplicates.

```
SELECT name FROM students WHERE age > 20  
UNION  
SELECT name FROM students WHERE course = 'CSE';
```

-- removes duplicates

```
SELECT name FROM students WHERE age > 20  
UNION ALL  
SELECT name FROM students WHERE course = 'CSE';
```

-- keeps duplicates

52. INTERSECT

- Returns only common rows from both queries.

```
SELECT name FROM students WHERE age > 20  
INTERSECT  
SELECT name FROM students WHERE course = 'CSE';
```

-- common rows only

53. EXCEPT (MINUS)

- Returns rows from first query that are not in second.

```
SELECT name FROM students WHERE age > 20  
EXCEPT  
SELECT name FROM students WHERE course = 'CSE';
```

-- not in second query

54. COMPARISON OPERATORS

OPERATOR	DESCRIPTION
=	Equal to
<> or !=	Not equal to
>	Greater than
<	Less than
>=	Greater than or equal to
<=	Less than or equal to

EXAMPLE

```
SELECT * FROM students  
WHERE age >= 21  
AND age <= 25;
```

55. LOGICAL OPERATORS

OPERATOR	DESCRIPTION
AND	Returns true if both conditions are true
OR	Returns true if any one condition is true
NOT	Reverses the result
IN	True if value matches any value in list
BETWEEN	True if value is within a range

EXAMPLE

```
SELECT * FROM students  
WHERE course = 'CSE'  
AND age BETWEEN 18 AND 25;  
  
SELECT * FROM students  
WHERE name IN ('Ravi', 'Aya', 'Arjun');  
  
SELECT * FROM students  
WHERE NOT course = 'ECE';
```

56. STRING FUNCTIONS

FUNCTION	DESCRIPTION
CONCAT(a, b)	Concatenates two strings
LENGTH(str)	Returns length of string
UPPER(str)	Converts to upper case
LOWER(str)	Converts to lower case
TRIM(str)	Removes leading/trailing spaces
REPLACE(str, a, b)	Replaces a with b
SUBSTRING(str, n, l)	Extracts substring

EXAMPLE

```
SELECT CONCAT(first_name, ' ', last_name)  
FROM students;  
  
SELECT UPPER(name) FROM students;  
  
SELECT LENGTH(name) FROM students;  
  
SELECT REPLACE(name, ' ', 'A')  
FROM students;
```

57. DATE & TIME FUNCTIONS

FUNCTION	DESCRIPTION
CURRENT_DATE()	Returns current date
CURRENT_TIME()	Returns current time
CURRENT_TIMESTAMP()	Returns current date & time
YEAR(date)	Extracts year
MONTH(date)	Extracts month
DAY(date)	Extracts day
DATE_ADD(date, INTERVAL n DAY)	Adds n days
DATE_SUB(date, INTERVAL n DAY)	Subtracts n days
DATEDIFF(date1, date2)	Returns difference in days

EXAMPLE

```
SELECT CURRENT_DATE();  
  
SELECT YEAR(date) FROM students;  
  
SELECT DATE_ADD(CURRENT_DATE(),  
INTERVAL 9 DAY);  
  
SELECT DATEDIFF('2024-06-01',  
'2024-05-23');
```

58. CASE WHEN (Conditional Logic)

- Used to perform conditional logic in SELECT statements.

```
SELECT name, age,  
CASE  
WHEN age <= 18 THEN 'Adult'  
WHEN age >= 19 THEN 'Teenager'  
ELSE 'Child'  
END AS age_group  
FROM students;
```

-- categorizes based on condition

59. COALESCE() & NULLIF()

- COALESCE() - Returns first non-NULL value.

```
SELECT COALESCE(middle_name, 'N/A')  
AS middle_name  
FROM students;
```

- NULLIF() - Returns NULL if values are equal.

```
SELECT NULLIF(discount, 0) AS disc  
FROM orders;
```

60. SELF JOIN

- A table joined with itself.

```
SELECT s.name AS student, f.name AS friend  
FROM students s  
JOIN students f ON s.friend_id = f.id;
```

-- student and their friend

61. CROSS JOIN

- Returns Cartesian product of two tables.

```
SELECT * FROM A  
CROSS JOIN B;
```

-- every row of A with every row of B

62. NATURAL JOIN

- Automatically joins on columns with the same name and removes duplicate columns.

```
SELECT * FROM students  
NATURAL JOIN courses;
```

-- joins on common column name

63. USING CLAUSE

- Used in JOIN to specify the common column.

```
SELECT * FROM students  
JOIN courses USING(course_id);
```

-- joins on course_id

64. EXISTS / NOT EXISTS

- Checks existence of rows in a subquery.

```
SELECT name FROM students s  
WHERE EXISTS (SELECT 1  
FROM orders o  
WHERE s.student_id = o.id);
```

-- students who have orders

```
SELECT name FROM students s  
WHERE NOT EXISTS (SELECT 1  
FROM orders o  
WHERE s.student_id = o.id);
```

-- students who have no orders

65. ANY, ALL

- ANY - True if condition matches any value.
- ALL - True if condition matches all values.

```
SELECT name FROM students  
WHERE age > ANY (SELECT age FROM students WHERE course = 'CSE');  
  
SELECT name FROM students  
WHERE age > ALL (SELECT age FROM students WHERE course = 'CSE');
```

66. WINDOW FUNCTIONS

- Performs calculation across a set of table rows related to current row.

FUNCTION	DESCRIPTION
ROW_NUMBER()	Assigns unique sequential number
RANK()	Gives rank for ties, leaves gaps
DENSE_RANK()	Gives rank for ties, no gaps
LAG(col, n)	Access data from n rows before
LEAD(col, n)	Access data from n rows after

EXAMPLE

```
SELECT name, age,  
ROW_NUMBER() OVER (ORDER BY age DESC)  
AS row_num  
FROM students;
```

67. COMMON TABLE EXPRESSION (CTE)

- A temporary result set that exists within the execution scope of a single SELECT, INSERT, UPDATE, or DELETE.

```
WITH high_students AS (  
SELECT name, marks FROM students WHERE marks > 80  
)  
SELECT * FROM high_students ORDER BY marks DESC;
```

-- temporary named result set

68. PIVOT (Using Aggregation)

- Transforms rows into columns.

```
SELECT course,  
SUM(CASE WHEN grade = 'A' THEN 1 ELSE 0 END) AS A_count,  
SUM(CASE WHEN grade = 'B' THEN 1 ELSE 0 END) AS B_count  
FROM results  
GROUP BY course;
```

-- pivot using CASE + SUM

69. UNPIVOT (Using UNION ALL)

- Transforms columns into rows.

```
SELECT id, 'Math' AS subject, math_marks AS marks FROM results  
UNION ALL  
SELECT id, 'Physics' AS subject, physics_marks AS marks FROM results  
UNION ALL  
SELECT id, 'Chemistry' AS subject, chem_marks AS marks FROM results;
```

-- unpivot using UNION ALL

70. TRANSACTION CONTROL (TCL)

COMMAND	DESCRIPTION
COMMIT	Saves all changes permanently
ROLLBACK	Undoes changes since last COMMIT
SAVEPOINT	Sets a point within a transaction
ROLLBACK TO	Rolls back to a savepoint
SET TRANSACTION	Sets transaction properties

EXAMPLE

```
START TRANSACTION;  
UPDATE accounts SET balance = balance + 100  
WHERE id = 1;  
SAVEPOINT S1;  
UPDATE accounts SET balance = balance + 100  
WHERE id = 2;  
ROLLBACK TO S1;  
COMMIT;
```

Practice more queries and build real projects to master SQL! 😊